LAB 04

Simple Python Database Connections

Practical Work and Screenshot Evidence

Student	Amna Pervez
Registration	2023-BSCS-10
Section	A
Course	CS-649 Cloud DevOps
GitHub	amnapervez8910

PROJECT LINKS

Programming fork

https://github.com/amnapervez8910/Databases

Submission directory

https://github.com/amnapervez8910/DevOps/tree/main/Labs/Lab04

Practical Overview

Tasks 1–7: setup, MySQL, MongoDB, environment variables and credentials-file configuration.

Execution environment

Ubuntu 26.04.1 LTS, accessed over SSH as amnapervez8910@ubuntu. Python executed on Ubuntu inside .venv. Docker hosted MySQL and the three MongoDB environments.

Compatibility adjustment

The supplied mongo:8.0 image stopped during startup with a Linux-kernel compatibility message. MongoDB 7.0.45 was used via compose.override.yaml with new MongoDB 7 volumes. The instructor-supplied compose.yaml was not edited. This differs from the supplied image version; instructor acceptance of the substitution has not been confirmed.

Verified connection targets

MySQL — school_mysql — 127.0.0.1:3307 — server environment dev

MongoDB dev — school_dev — 127.0.0.1:27017 — dev

MongoDB test — school_test — 127.0.0.1:27018 — test

MongoDB simulated prod — school_prod — 127.0.0.1:27019 — prod

Repository and credential handling

The same db_connect.py was edited through all four stages. The final credentials-file version and Compose override were pushed in commit e677c4a. credentials.ini and .venv were ignored, and credentials.ini was confirmed untracked. The local credentials file was set to mode 600. After evidence capture, the databases were stopped with docker compose down, preserving volumes.

Task 1 — Fork and Clone — Own Databases Fork

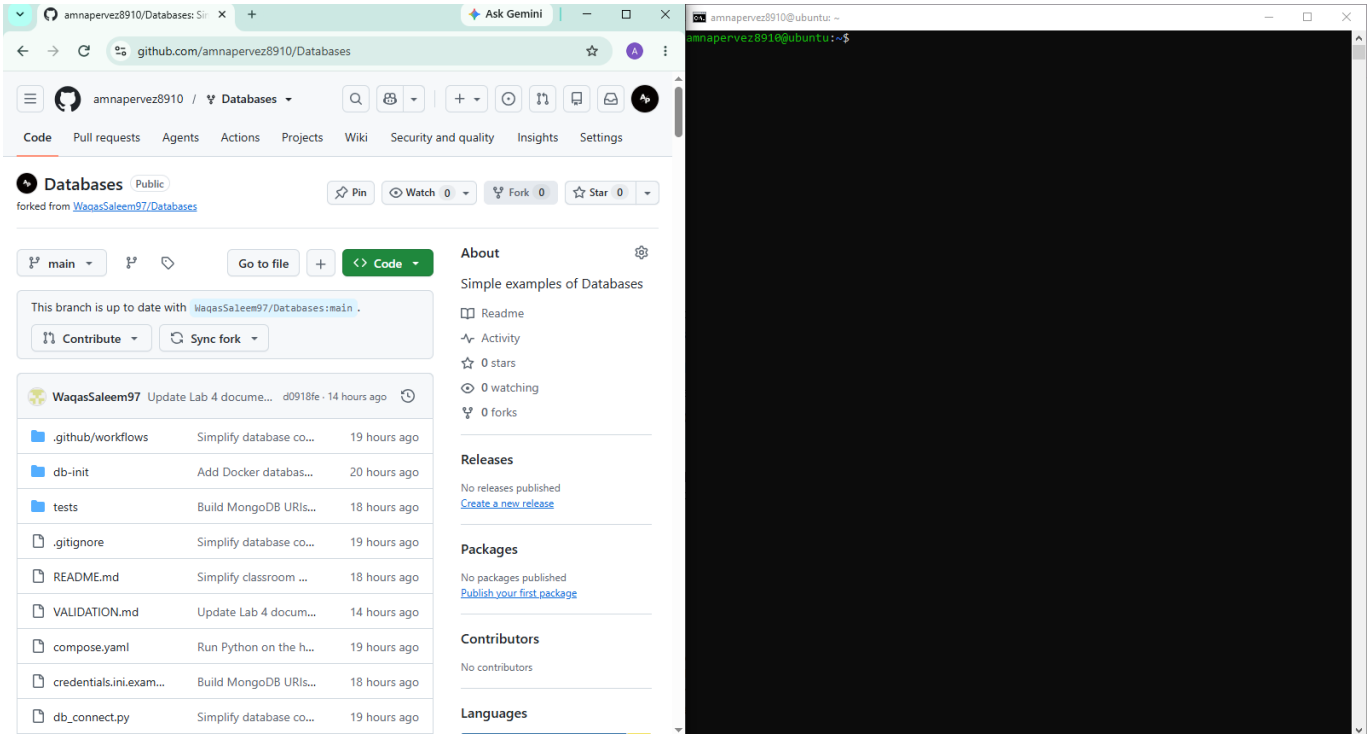


Figure 01 — databases_fork.png

Task 1 — Fork and Clone

SSH clone completed

The SSH clone completed successfully. The directory listing shows the supplied Python file, Compose configuration, requirements and initialization scripts.

The working directory for the programming tasks is ~/Databases.

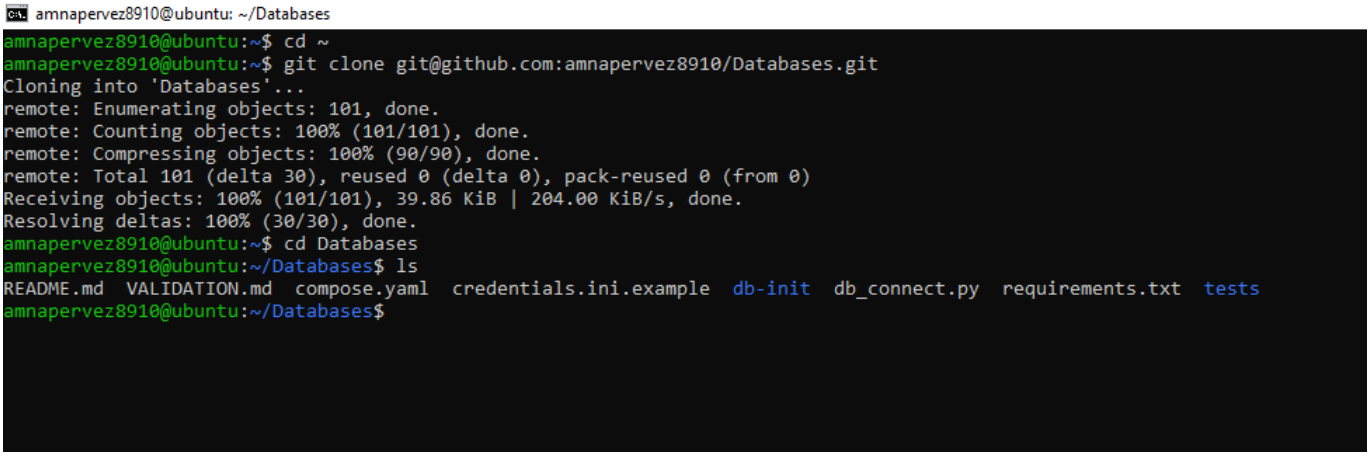


Figure 02 — databases_clone.png

Task 2 — Install Docker, Compose and Python

Installed tool versions

Git 2.53.0, Docker 29.1.3 and Docker Compose v5.0.0 are visible in the terminal output.

Python ran directly on Ubuntu in an activated .venv; Docker ran the database services.

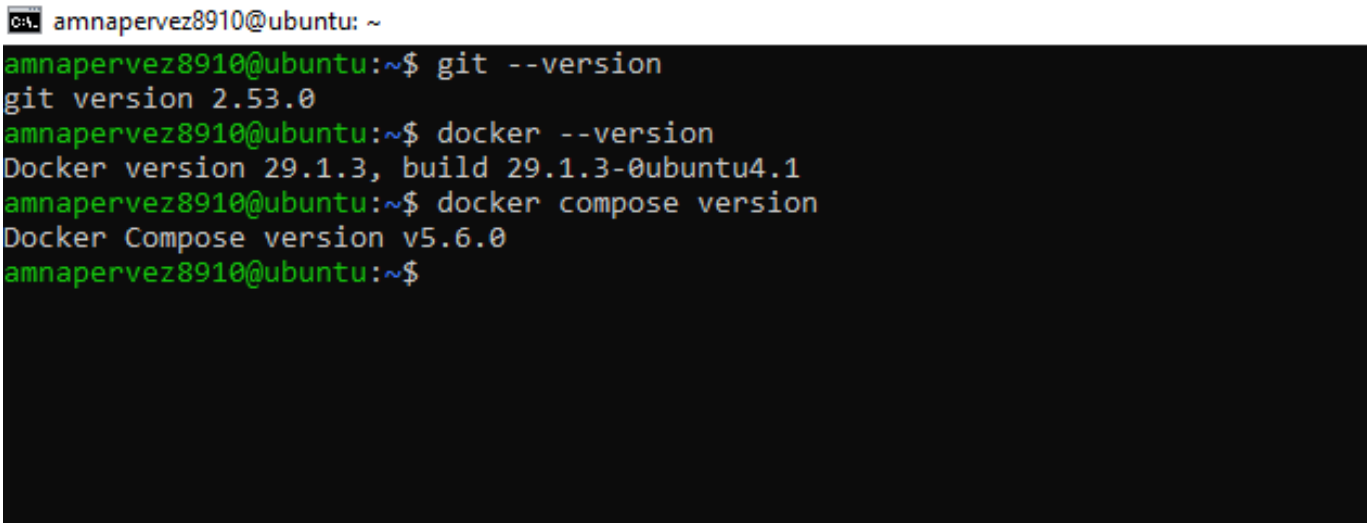


Figure 03 — docker_installation.png

Task 3 — Start Database Services

Four healthy database services

docker compose ps shows MySQL and the three MongoDB services running with healthy status.

Compatibility note: MongoDB 7.0 was used through compose.override.yaml after the supplied 8.0 image refused startup on this Ubuntu kernel. Original compose.yaml was retained; separate MongoDB 7 volumes were used.

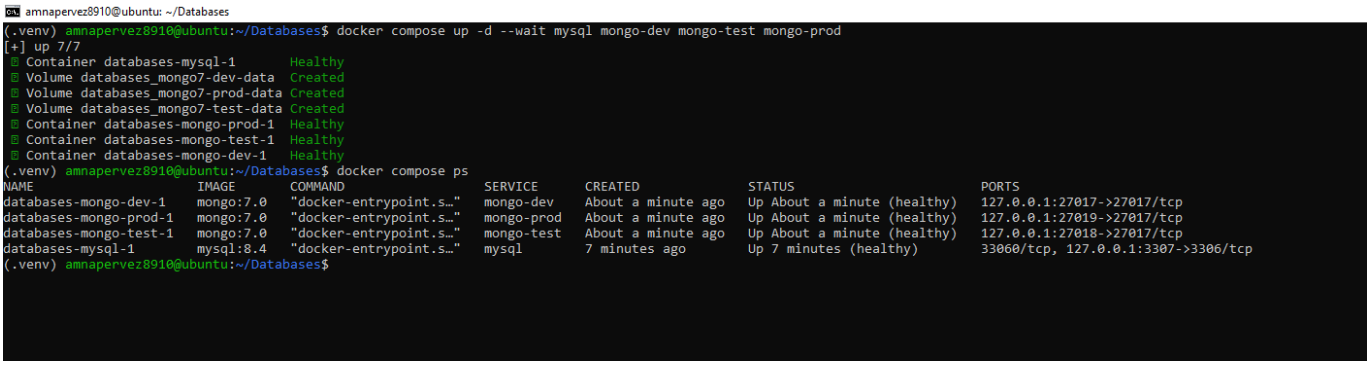


Figure 04 — database_services.png

Task 3 — Start Database Services

Published database ports

MySQL is exposed at 127.0.0.1:3307. MongoDB dev, test and simulated prod use ports 27017, 27018 and 27019 respectively.

Python runs on the Ubuntu host, so it connects to the published host ports rather than Docker service names.

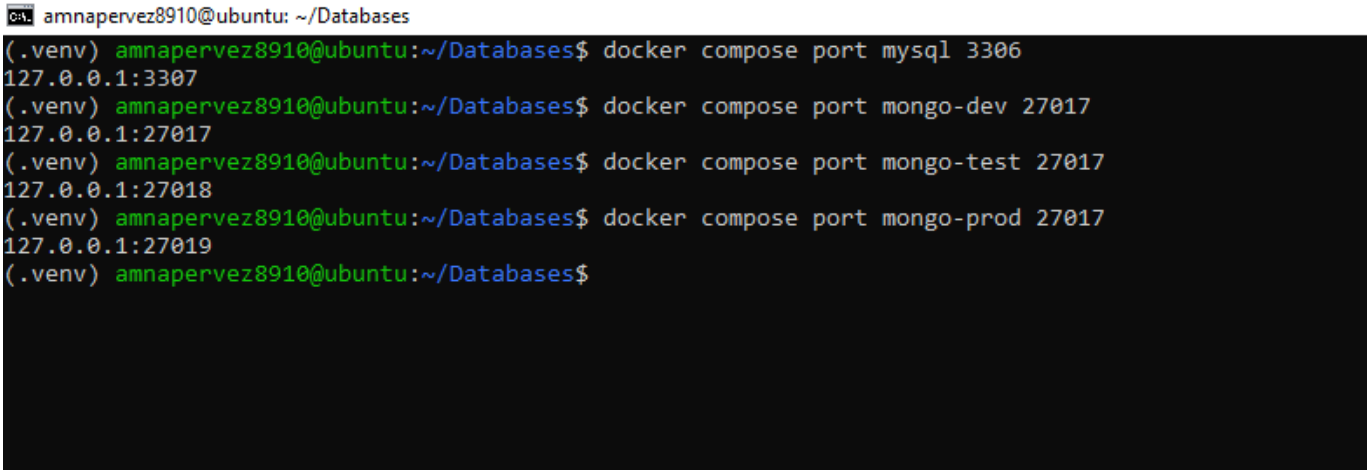


Figure 05 — database_ports.png

Task 4 — MySQL with Values in Code

MySQL connection verified

The program connected to school_mysql at 127.0.0.1:3307 and read Environment: dev from the server.

mysql.connector.connect() opens the connection. The SQL query reads the actual environment value; the cursor and connection are then closed.

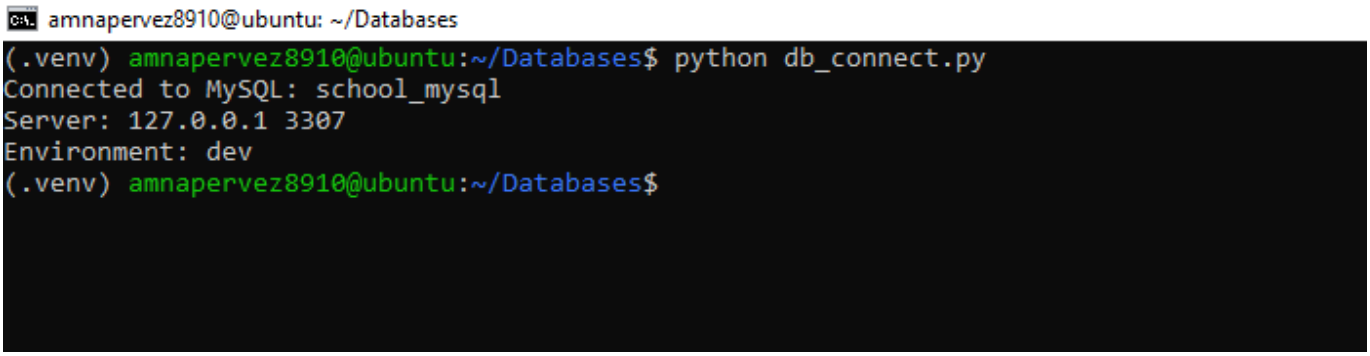


Figure 06 — mysql_hardcoded_connection.png

Task 5 — MongoDB with Values in Code

Hardcoded MongoDB connection verified

The same db_connect.py connected to school_dev at 127.0.0.1:27017 and read Environment: dev.

The URI combines the username, password, endpoint and database name. ping sends a real request, and environment_info confirms the target. No credential-bearing URI is printed.

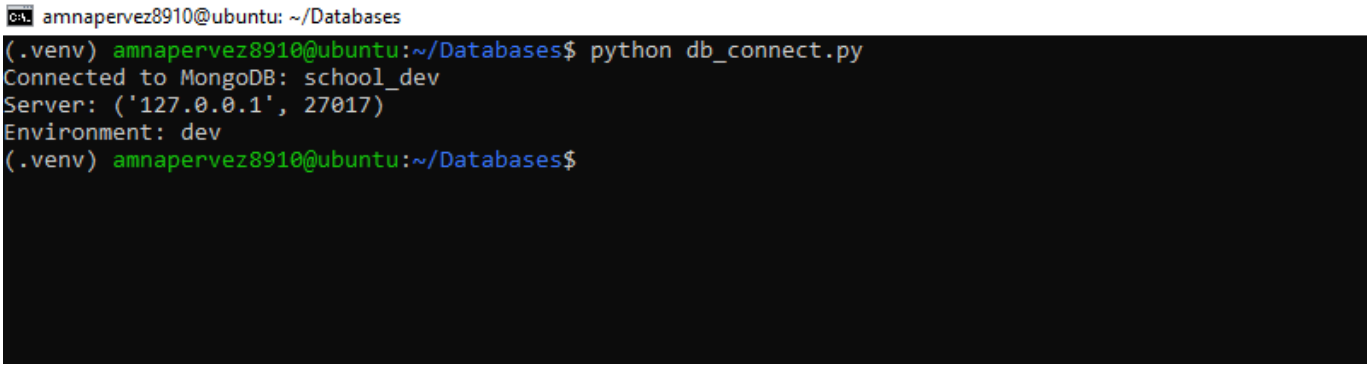


Figure 07 — mongo_hardcoded_connection.png

Task 6 — MongoDB with os.getenv()

Environment-variable version: dev

With the four dev variables exported, the unchanged program connected to school_dev on port 27017 and read Environment: dev.

os.getenv() reads MONGO_USERNAME, MONGO_PASSWORD, MONGO_ENDPOINT and MONGO_DATABASE. All four matching settings select the target; this stage does not use APP_ENV.

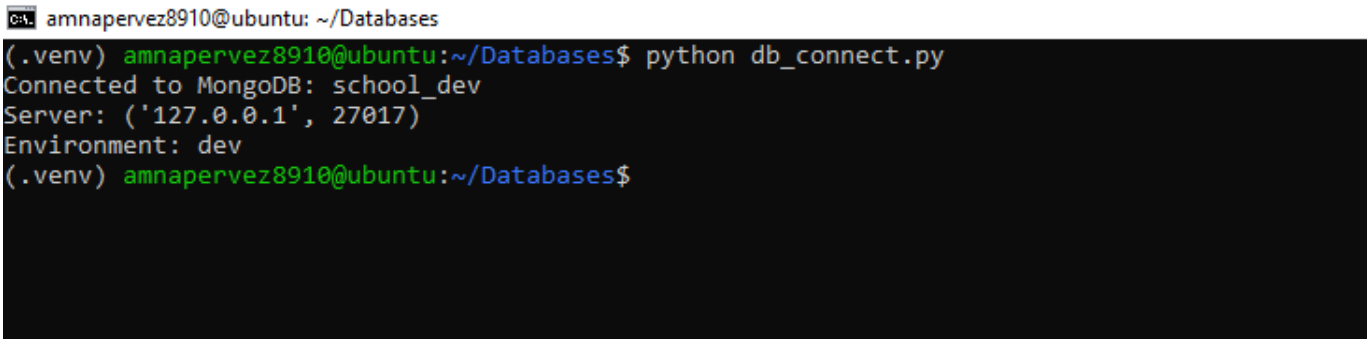


Figure 08 — mongo_env_dev.png

Task 6 — MongoDB with os.getenv()

Environment-variable version: test

With the test variables exported, the program connected to school_test on port 27018 and read Environment: test.

Only the shell settings changed between runs; the Python program and MongoDB connection block remained unchanged.

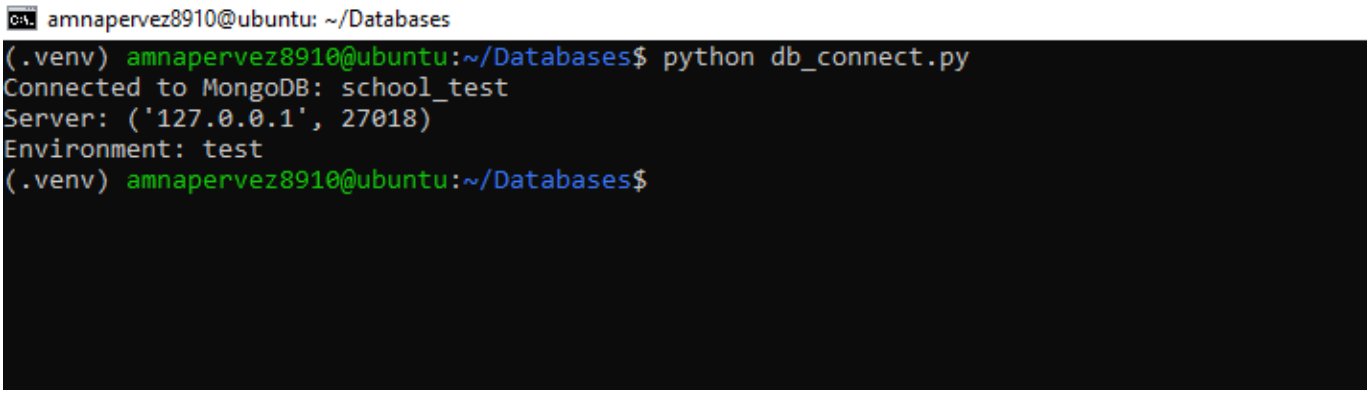


Figure 09 — mongo_env_test.png

Task 6 — MongoDB with os.getenv()

Environment-variable version: simulated prod

With the prod variables exported, the program connected to school_prod on port 27019 and read Environment: prod.

The prod environment is a disposable local classroom simulation, not a production service.

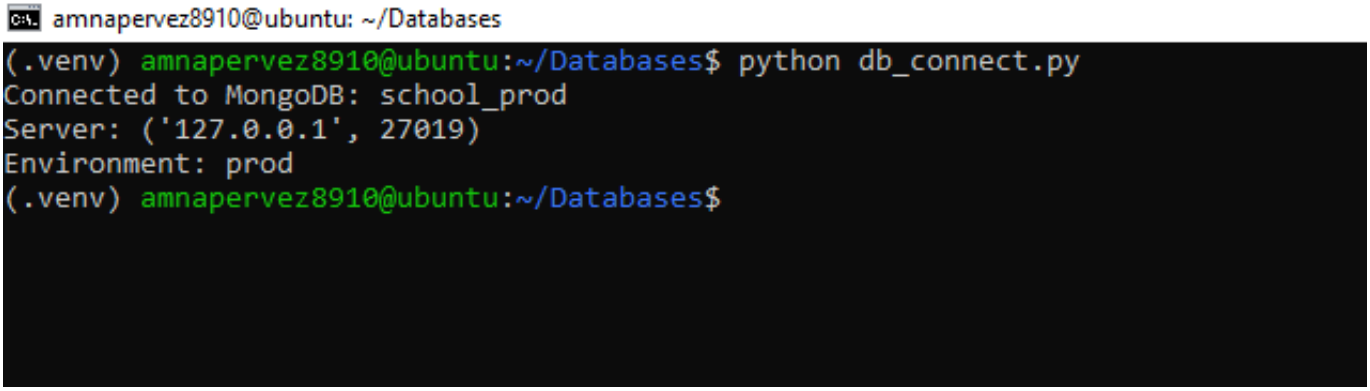


Figure 10 — mongo_env_prod.png

Task 7 — MongoDB with a Credentials File

Local credentials file created

credentials.ini was copied from credentials.ini.example, and ls confirmed that the local file exists.

The cp -n portability warning did not prevent file creation. The credential file contents are not shown in the screenshot and were not committed.

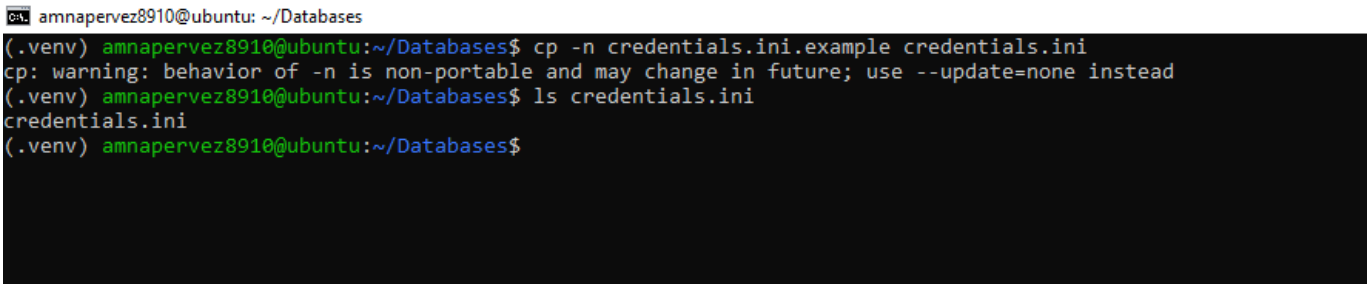


Figure 11 — credential_files_created.png

Task 7 — MongoDB with a Credentials File

Owner-only credential file permissions

chmod 600 credentials.ini was applied. stat confirms 600 credentials.ini.

Mode 600 gives the file owner read/write access and grants no access to group or other users. It does not encrypt the plaintext file.

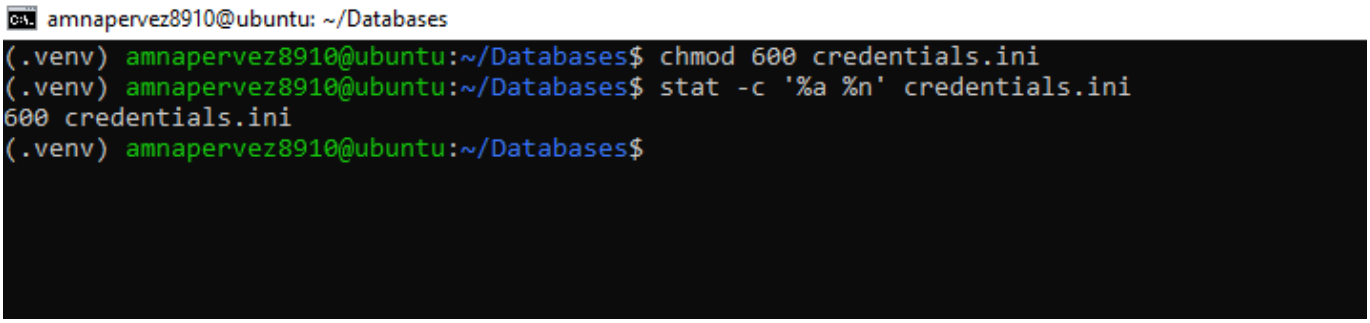


Figure 12 — credential_file_permissions.png

Task 7 — MongoDB with a Credentials File

Credentials-file version: dev

APP_ENV=dev selected the dev section. The program connected to school_dev on port 27017 and read Environment: dev.

configparser reads credentials.ini. APP_ENV selects a section, defaulting to dev if unset. The connection block is the same as in the previous MongoDB stages.

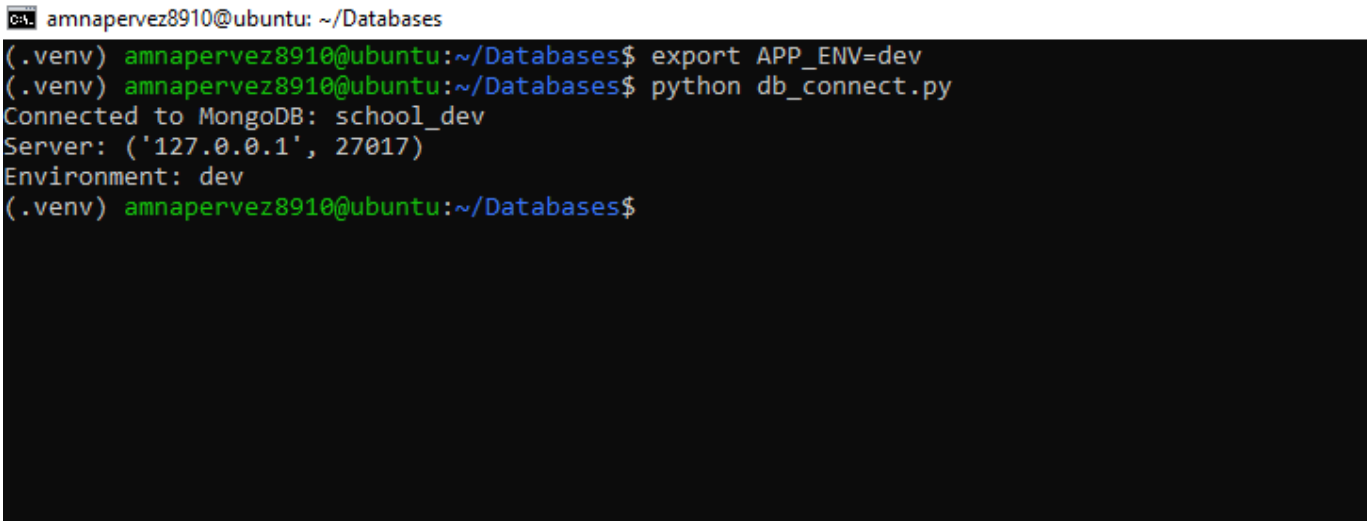


Figure 13 — mongo_file_dev.png

Task 7 — MongoDB with a Credentials File

Credentials-file version: test

APP_ENV=test selected the test section. The program connected to school_test on port 27018 and read Environment: test.

No Python edits were needed to switch environments. The four connection settings were read from the selected INI section.

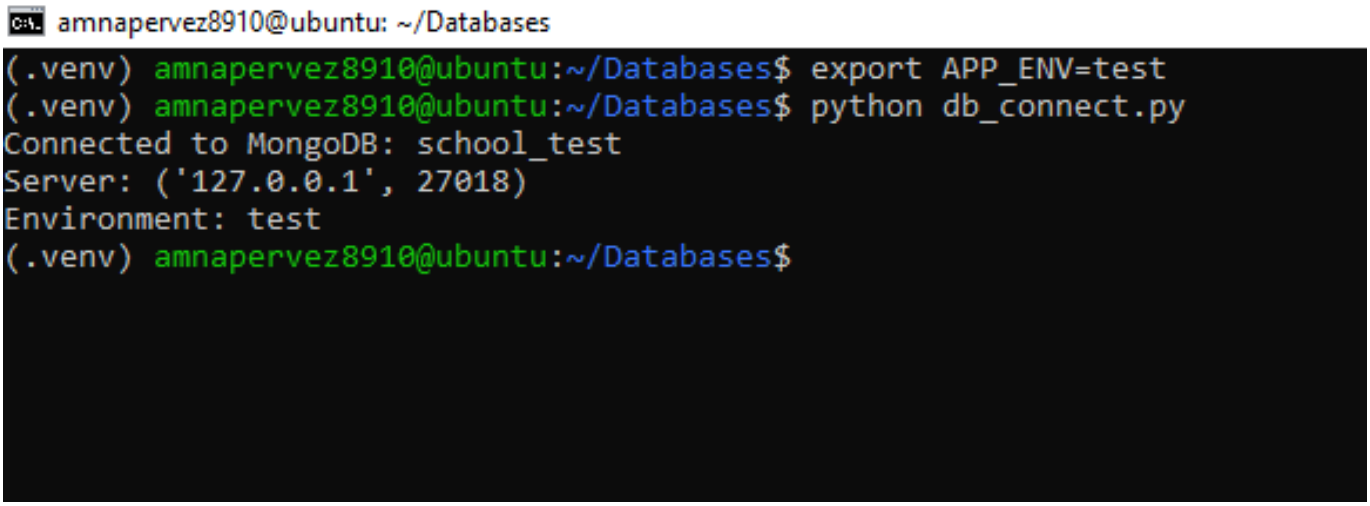


Figure 14 — mongo_file_test.png

Task 7 — MongoDB with a Credentials File

Credentials-file version: simulated prod

APP_ENV=prod selected the prod section. The program connected to school_prod on port 27019 and read Environment: prod.

The credentials-file version was retained as the final db_connect.py. The code and Compose override were pushed to the Databases fork; docker compose down was then used without -v.

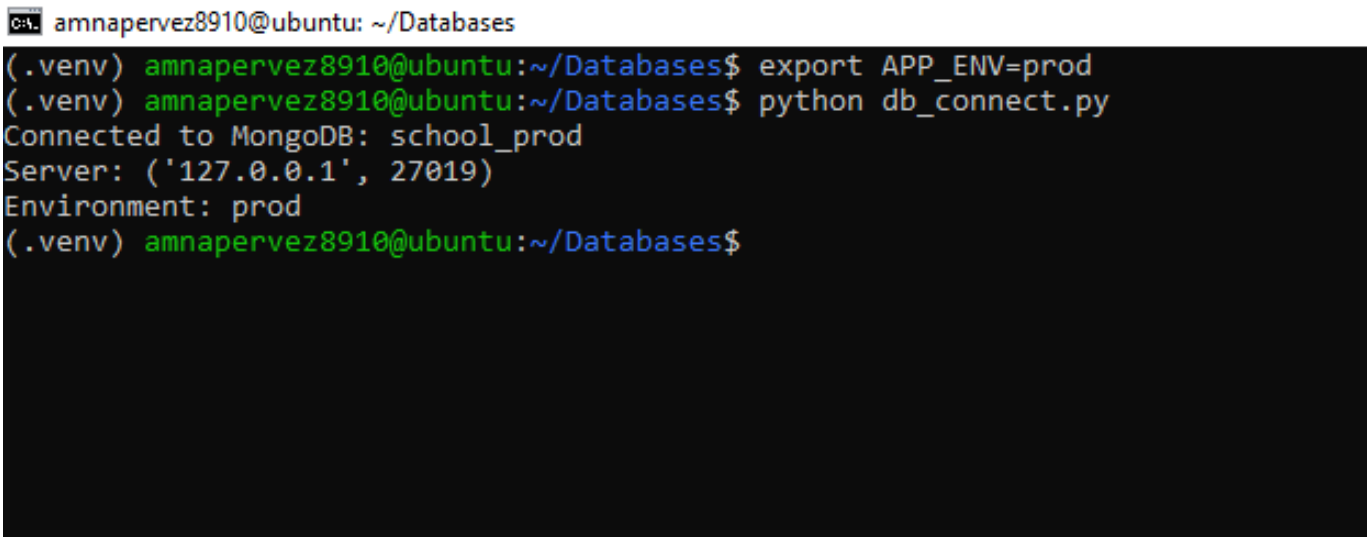


Figure 15 — mongo_file_prod.png